

PROYECTO 2:

SISTEMA MULTIMODAL DE RECUPERACIÓN Y BÚSQUEDA

1. Introducción

En este proyecto diseñarás e implementarás un sistema unificado de recuperación y búsqueda que funciona sobre múltiples modalidades (texto, imágenes, audio) bajo una arquitectura común. El objetivo es entender cómo diferentes tipos de contenido pueden indexarse y recuperarse mediante el mismo paradigma fundamental:

- Dividir (split) el contenido en unidades atómicas (chunks).
- Extraer características (features/descriptores) adaptados a cada modalidad.
- Construir un diccionario compartido (codebook) mediante técnicas específicas de cada modalidad.
- Generar índices invertidos que mapean palabras/conceptos a chunks.
- Comparar tu implementación (índices invertidos + codebook) con técnicas nativas de PostgreSQL: GIN/GiST para texto, pgvector para imágenes y audio.
- Implementar al menos dos de las aplicaciones sugeridas para interactuar con el motor de búsqueda multimodal.
- Analizar diferencias en rendimiento, precisión y trade-offs entre enfoques.

Objetivos de Aprendizaje

- Entender la arquitectura unificada: cómo split + codebook + índice invertido se aplica de manera agnóstica a modalidades.
- Analizar diferencias técnicas: GIN (Generalized Inverted Index) vs. tu índice invertido; pgvector (búsqueda vectorial aproximada) vs. codebook + histogramas.
- Medir y comparar: latencia, precisión (recall), impacto de dimensionalidad, accesos a disco en cada enfoque.
- Identificar trade-offs: exactitud vs. velocidad, memoria vs. latencia, simplicidad vs. sofisticación.
- Justificar decisiones de diseño basadas en evidencia experimental rigurosa.

2. Especificación Funcional

Arquitectura Unificada Multimodal

Tu sistema implementará una arquitectura común aplicable a texto, imágenes y audio:



Fases del Proyecto

Fase 1: Análisis, Diseño y Selección del Dataset

- Especificar requisitos funcionales (modalidades soportadas, tipos de consultas) y no-funcionales (latencia, precisión).
- Seleccionar dataset público. Documentar: tamaño, características, distribución.
- Diseñar arquitectura: adaptación de split, extracción y codebook a cada modalidad.
- Definir métricas: latencia (ms), throughput (consultas/seg), precisión, memoria y accesos a disco.
- Seleccionar al menos dos aplicaciones para implementar en el backend, sustentar importancia.

Fase 2: Implementación del Sistema Multimodal

- Módulo Split: dividir contenido (párrafos para texto, descriptores locales para imágenes, ventanas deslizantes para audio).
- Módulo Extractor: TF-IDF para texto, SIFT para imágenes, MFCC para audio.
- Módulo Codebook: técnicas lingüísticas para texto (tokenizar, eliminar stopwords, stemming, top-k); K-Means/K-Medoids para imágenes y audio.
- Módulo Índice Invertido: para cada chunk, generar histograma de codewords. Para Texto implementar el SPIMI.
- Backend extensible: arquitectura que soporte al menos dos aplicaciones diferentes (RAG para texto, búsqueda de imágenes, búsqueda de música, etc.).
- Persistencia: almacenar en PostgreSQL (codebook, histogramas, metadatos, etc.).

Fase 3: Implementación de Comparativas en PostgreSQL

- GIN/GiST (Texto): índices nativos PostgreSQL. Ejecutar consultas full-text. Medir latencia, memoria.
- pgvector (Imagen/Audio): almacenar histogramas como vectores. Crear índice HNSW o IVF. Medir latencia, precisión, memoria.

Fase 4: Evaluación Experimental y Análisis

- Marco: cargas variadas (pequeña: 1K chunks, mediana: 10K, grande: 100K).
- Métricas: latencia, throughput, precisión, memoria, accesos I/O.
- Comparar: tu índice invertido vs. GIN/GiST vs. pgvector. Analizar trade-offs, generar gráficos comparativos.

Conclusiones: ¿Qué técnica ganó en qué métrica? ¿Se recuperó la misma información (precisión)?, limitaciones y recomendaciones.

3. Consideraciones Técnicas

Construcción del Codebook por Modalidad

Texto: Técnicas lingüísticas:

- Tokenización → Normalización (conversión a minúsculas, eliminación de puntuación) → Eliminación de stopwords → Stemming o Lemmatización
- Selección: tomar las k palabras más frecuentes en toda la colección. Estas k palabras conforman el diccionario (codebook) textual.

Imágenes: Clustering sobre descriptores locales:

- Extracción: SIFT por patch/región.
- Agrupación: K-Means sobre todos los SIFT de todas las imágenes → k clusters.
- Codebook: k centroides representan "palabras visuales" (visual words).

Audio: Clustering sobre características acústicas:

- Extracción: MFCC con sliding window.
- Agrupación: K-Means sobre todos los MFCC de todos los audios → k clusters.
- Codebook: k centroides representan "palabras acústicas" (acoustic words).

Tu Implementación: Índice Invertido + Codebook

- Cuantización (imagen/audio): reduce descriptores altos a k codewords (compresión lossy).
- Inversión lingüística (texto): mapea top-k palabras a chunks para búsqueda rápida.
- Compacta: cada chunk es solo un vector de frecuencias (tiny vectors).
- Agnóstica: mismo algoritmo base para todas las modalidades.
- Ventaja: ultra-rápida, bajo consumo RAM, conceptualmente simple.
- Desventaja: cuantización introduce falsos positivos/negativos; calidad depende de k; no captura todas las variaciones.

4. Informe, Presentación y Entregables

- Repositorio GitHub con código fuente bien documentado.
- Docker Compose: PostgreSQL + pgvector, scripts de inicialización.
- Informe técnico conciso en README o Wiki de GitHub (no extenso, pero completo, con gráficos ilustrativos):
 - Descripción del sistema y arquitectura.
 - Dataset utilizado y características.
 - Detalles de implementación por módulo.
 - Resultados experimentales: tablas y gráficos comparativos.
 - Análisis de trade-offs y conclusiones.
 - Instrucciones de instalación y uso.
 - Cuidado en ortografía, redacción técnica y consistencia de párrafos.
- Ejemplos de uso: casos de aplicación implementados (al menos dos de Ideas Sugeridas).
- Presentación oral con demostración viva de al menos dos aplicaciones funcionando.
- GitHub Projects: planificación y seguimiento con hitos alcanzados.

Equipos: 5 integrantes por proyecto.

Evaluación de Planificación: Se evaluará división clara de tareas, hitos cumplidos a tiempo, cierre de issues documentado, equilibrio de contribuciones, evidencia de colaboración.

Ideas de Aplicaciones Sugeridos

Cuatro opciones de sistemas multimodales. Todos utilizan la arquitectura unificada (split + codebook + índice invertido). Tu backend debe soportar al menos dos de estas aplicaciones.

1. Búsqueda Visual E-commerce (Modalidad Primaria: Imagen)

Usuario sube foto, sistema retorna 10 productos similares. Búsqueda por imagen: "encontrar prendas similares a esta foto".

- Split: descriptores locales, patches superpuestos o región de imagen.
- Extracción: SIFT o Inception V3.
- Codebook: K-Means sobre SIFT → diccionario visual (visual words).
- Índice: histograma de visual words por producto.

Dataset: Fashion Product Images Dataset (Kaggle) - 44,000 imágenes de productos de moda.

2. Búsqueda Musical Inteligente (Modalidad Primaria: Audio + Texto)

Buscar canciones: por letra (full-text), características acústicas, o similitud acústica. "Encuentra canciones con ritmo similar y en géneros relacionados".

- Split (Texto): párrafos de letras; Split (Audio): ventanas de 100-200ms.
- Extracción: TF-IDF para letras; MFCC para audio.
- Codebook: lingüístico para texto (top-k palabras); K-Means para audio.
- Índice: histogramas texto + audio por canción.

Dataset: Audio features and lyrics of Spotify songs (Kaggle) o FMA: A Dataset For Music Analysis (GitHub).

3. Búsqueda Multimodal en Documentos (Modalidad Primaria: Texto + Imagen)

Indexa artículos/papers con texto e imágenes incrustadas. Busca por texto o similitud visual: "¿Qué artículos mencionan machine learning con imágenes relacionadas?".

- Split: párrafos de texto; patches de imágenes.
- Extracción: TF-IDF para párrafos; SIFT para patches de imágenes.
- Codebook: lingüístico para texto; K-Means para imagen.
- Índice: histogramas texto + imagen por documento.

Dataset: BBC News con imágenes, ArXiv abstracts, o corpus académico con figuras.

4. Recomendación Multimodal (Modalidad Primaria: Imagen + Descripción)

Recomienda productos similares: "si te gusta este producto, también te podría gustar...".

- Split: patches de imagen; párrafos de descripción de producto.
- Extracción: SIFT para imágenes; TF-IDF para descripciones.
- Codebook: K-Means para imagen; lingüístico para descripción.

- Índice: histogramas visuales + textuales por producto.
- Fusión: combina scores de similitud visual + textual para ranking final.

Dataset: Fashion Product Images Dataset (Kaggle).